

How the v5 HSMM footprint model works (conceptual explainer)

Companion to `classify_single_molecule_binding_v5_hsmm.py`, `SPEC_hmm_footprint_model.md` (the design), and `NOTES_v5_hsmm_status.md` (status + full parameter reference). This file is the *why/how* — the intuition and the math — not the API.

1. The problem we’re solving

Each molecule is a row of an SMF matrix: at every GpC position we observe 1 (protected — enzyme could not methylate, so something was bound), 0 (accessible — methylated), or -1 (no data). We want to explain each molecule as a linear arrangement of **footprints**: stretches of protection caused by nucleosomes or bound TFs, separated by accessible linker.

The hard part is that the same observed pattern can be explained many ways (a long protected stretch = one big nucleosome? two nucleosomes? a nucleosome plus a TF?), and the “right” answer depends on *shape and context*: a TF is a short punctate blob bounded by accessible DNA; a nucleosome is a ~147 bp footprint with soft edges; adjacent things behave differently than isolated ones. We need a model whose notion of “footprint” is native, not bolted on.

2. Two ways to model this: v4 (enumerate) vs v5 (segment)

v4 — enumerate every global microstate

v4 asks: *of all the physically-valid whole-molecule configurations (every combination of nucleosome-dyad placements x every subset of TFs bound, minus overlaps), which one makes this molecule most likely?* It literally builds the list of all such microstates, scores the molecule against each, and takes the best.

This works, but: - The state list explodes combinatorially and has to be pruned and de-duplicated (“collapse”). - The scoring is **site-independent** — each GpC is judged on its own, and the channels (nuc/TF/ open) are combined by **max**. There is **no representation of a “run” or a “boundary.”** So facts like “a real TF is bounded by accessible DNA on both sides” or “the base between two adjacent TFs is usually but not always accessible” can’t be *stated* by the model — they have to be added back as **penalty terms** (the TF-boundary penalty, the run-length Occam cap). That’s the hack accretion the spec set out to end.

v5 — segment the molecule (hidden semi-Markov model)

v5 asks a different question: *what is the most likely left-to-right **sequence of segments** that tiles this molecule?* A segment is (type, start, end). The molecule is explained as, e.g., `OPEN[0--80] * NUC[80--225] * OPEN[225--278] * TF[278--304] * OPEN[304--318] * TF[318--344] * ...`

Nothing is enumerated globally. The model is **generative and local**: it has rules for how one segment follows another, how long each type tends to be, and what each type emits. The single best tiling is found by dynamic programming (Viterbi). Every v4 penalty becomes a *structural* part of the model instead of an add-on (see Sec.7).

3. What “hidden semi-Markov” means

- **Hidden**: we never observe the footprints directly — we infer (“decode”) them from the GpC readout. The footprint sequence is the hidden/latent variable.
- **Markov**: which segment type comes next depends only on the *current* segment type (not the whole history). That’s what makes efficient DP possible.

- **Semi-Markov (the “semi”)**: unlike a plain HMM — where the model re-decides the state at every single position and a “long state” is just the same state repeated with a geometric length — a *semi*-Markov model emits a whole **segment of an explicit duration at once**, and that duration has its own probability distribution. This matters enormously here: a nucleosome is ~147 bp *because that’s its physical size*, not because of a per-base “stay” probability. The semi-Markov duration model lets us say “NUC segments are ~147 bp long, TF segments are motif-width, UNID segments are short” directly.

4. The generative story (how the model “imagines” a molecule)

1. Start at position 0. Pick a first segment type.
2. Pick that segment’s **duration** from its duration distribution -> this sets where it ends.
3. The segment **emits** GpC observations across its span according to its emission model.
4. **Transition** to the next segment type (paying a transition cost), and repeat from step 2, until the molecule is fully tiled to the end.

Every molecule is a full, gapless tiling [0 ... amp_width). The four segment types:

Type	Where	Duration	“Truly protected” prob p_U
OPEN	anywhere	flexible (linker)	low (0.05) — accessible
NUC	dyad anywhere	Gaussian ~147 bp (100–210)	position-dependent : sigmoid, high near dyad, decaying on flanks
TF	only spanning an annotated motif	fixed = motif width (+margin)	flat high (0.9)
UNID	off-motif, non-nuc length only	short (15–90 bp)	flat high (0.9)

UNID (“unidentified footprint”) is the genuinely new capability: a protected segment that is neither at a known motif nor nucleosome-shaped. It’s the model’s way to say “*there is clearly a footprint here, but I can’t name it.*” It carries the highest start cost so it only wins when nothing else fits (see Sec.7).

5. The likelihood — what score we’re maximizing

The score of one candidate segmentation of one molecule is a sum of four kinds of log-terms:

```
score(segmentation) = Sigma_segments [ emission(segment) + start_cost(type) + duration_logprob(type, 1
                        + Sigma_boundaries transition_cost(prev_type -> next_type)
```

5a. Emission — the only part that touches the data

Every state has a “truly protected” probability p_U. The DNA readout is noisy (incomplete conversion, etc.), so p_U is pushed through the **conversion model** (identical to v4) to get the probability of what we actually *observe*:

$$P(\text{observe protected} \mid \text{state, position}) = p_U * p_{\text{t_given_unmeth}} + (1 - p_U) * p_{\text{t_given_meth}}$$

with (in your runs) p_t_given_unmeth = 0.99 (a truly-protected base almost always reads protected) and p_t_given_meth = 0.05 (a truly-accessible base rarely reads protected). A segment’s emission is the sum of log-probabilities over the GpCs inside it:

```
emission(segment) = Sigma_{GpC j in segment} [ obs_j * log P(protected) + (1-obs_j) * log P(accessible)
```

- For **OPEN/TF/UNID**, p_U is a single flat number, so this is just “how many protected vs accessible GpCs fall in the window, weighted by log-probs.” Computed in $O(1)$ from cumulative sums.
- For **NUC**, p_U depends on distance from the dyad: $p_U(\text{pos}) = \text{sigmoid}((d_{\text{edge}} - |pos - dyad|)/\text{softness})$. This is the v4 nucleosome sigmoid, reused verbatim: near the dyad, methylation is strongly blocked (high p_U); on the flanks it decays. So a nucleosome *expects* protection in the middle and tolerates accessibility at the edges — that soft-edge shape is what distinguishes it from a flat TF.
- **Missing GpCs (-1) contribute nothing** — they’re skipped, not guessed. (This differs from v4, which soft-counted missing as 0.5.)

Intuition for the numbers: a *correctly* explained GpC adds $\sim \log(0.99) \sim -0.01$; a *badly* explained one (model says accessible, data says protected) adds $\sim \log(0.05) \sim -3.0$. So each mis-explained GpC costs about 3 “nats.” That’s the currency the structural costs below trade against.

5b. Start costs — parsimony

Every non-OPEN segment pays a fixed cost just to exist (`start_cost_nuc/tf/unid`). This stops the model from over-explaining noise: adding a footprint has to “pay for itself” by fixing more than ~ 1 mis-explained GpC. UNID has the **highest** start cost (6.0), which is precisely what makes it the explanation of last resort — a nucleosome or TF will win whenever it fits comparably well.

5c. Duration priors — footprint size/shape

NUC duration follows a Gaussian peaked at 147 bp (sigma ~ 25 , clamped to 100–210). This is what lets a long protected stretch be scored as **two nucleosomes** rather than one implausible 280 bp one — a single over-long NUC pays a steep duration penalty, while two ~ 140 bp NUCs each sit near the peak. TF duration is fixed (motif width). UNID is confined to a short band and, by construction, can’t be nucleosome-length. This *is* the v4 “run-length Occam cap,” but expressed as a proper length distribution instead of a bolt-on penalty.

5d. Transition costs — adjacency rules

`transition_cost(prev -> next)` encodes what may sit next to what, and how much it’s discouraged:

- Returning to OPEN is free.
- NUC $\rightarrow$ NUC (di-nucleosome, no linker), TF $\rightarrow$ TF (adjacent TFs, no linker), NUC $\leftrightarrow$ TF (a TF right against a nucleosome), and UNID adjacencies each cost a tunable amount.
- OPEN $\rightarrow$ OPEN and UNID $\rightarrow$ UNID are **disallowed** (a segment is already maximal — you can’t split one open stretch into two).
- A TF may only occupy an annotated motif; UNID may never overlap a motif. These are *hard* constraints, not costs.

`trans_nuc_tf` is the one most worth calibrating from real data (via `protection_streak_histogram.py`): it decides how readily the model puts a TF flush against a nucleosome.

6. Decoding — how we find the best segmentation (Viterbi)

We don’t score every possible segmentation (there are astronomically many). Instead, **segmental Viterbi** builds the answer up position by position with a dynamic-programming recursion:

$V[\text{end}, \text{type}]$ = the best possible score of any segmentation of $[0, \text{end})$ whose **last segment has this type**.

To fill in $V[\text{end}, \text{type}]$, we try every possible **start** for that last segment and every possible **previous** type it could have followed:

```
V[end, type] = max over (start, prev_type) of
    V[start, prev_type]                # best way to reach the start
    + transition_cost(prev_type -> type) # adjacency
    + start_cost(type) + duration_logprob(type, end-start) # this segment's structure
    + emission(start, end, type)        # this segment vs the data
```

We remember which (`start`, `prev_type`) won (a “backpointer”). After sweeping to the end, we read the backpointers backwards to recover the single best segment sequence — the **Viterbi path**.

Why this is cheap: the work is $O(L * S * D)$ per molecule ($L \sim 600$ positions on a 5 bp grid, $S=4$ states, D =duration options), and **molecules are independent**, so it’s embarrassingly parallel and vectorized across all molecules at once. No global microstate list, no pruning, no collapse. In practice the whole model runs on a boundary **grid** (default 5 bp spacing, plus every motif edge added exactly so TF segments land on motifs), which keeps it fast while staying precise where it matters.

The reported `log_likelihood` per molecule is the total score of its winning path. **Caveat:** it is *not* comparable to v4’s log-likelihood — v4 sums one global `max`-combined probability vector, while v5 sums per-segment emissions plus structural costs. They are different quantities.

6b. Where boundaries live: the grid vs. the GpCs (common point of confusion)

Two coordinate ideas are deliberately separate:

- **Geometry is on a bp grid.** Segment start/end/duration and nucleosome dyads live on a boundary grid: every `grid_step` bp (default 5) from 0 to the amplicon end, **plus every motif edge added exactly**. Transitions between states happen only at these grid points. `grid_step` is a pure speed/precision knob (finer = slower, quadratically); dyad resolution $\sim \text{grid_step}/2$. Motif edges are always on the grid, so TF placement is exact regardless of `grid_step`.
- **Emissions are only at GpCs.** The data likelihood is evaluated at GpC positions using their exact bp coordinates. Between GpCs there’s no data, so sliding a boundary within a GpC-free stretch changes only the duration prior, not the emission.

Consequences: - All length parameters (`nuc_min/max`, `unid_min/max`, durations) are **bp spans**, NOT GpC-to-GpC distances and NOT GpC counts. `unid_min=15` means the segment spans ≥ 15 bp; it says nothing about how many GpCs are inside. - A segment’s *length* is geometric but its *evidence* is only the GpCs inside it. A short segment containing zero GpCs emits nothing and can never win (it can’t pay its start cost). So in practice a UNID needs ≥ 1 protected GpC in its span to be called.

Why a uniform grid and not just GpC-midpoints? For the *flat* states (OPEN/TF/UNID) the emission is piecewise-constant in boundary position — it only changes when a boundary crosses a GpC — so the midpoints between consecutive GpCs would be a *sufficient* (and faster) boundary set. The reason for the finer uniform grid is the **nucleosome dyad**: the NUC emission depends *continuously* on the dyad (the sigmoid is centered at it, and dyad = segment center), so finer boundaries give finer dyad placement and a genuinely better fit; the duration prior is likewise continuous in length; and GpC spacing is uneven (a few bp to ~ 40 bp gaps), so a uniform grid gives consistent dyad/duration resolution everywhere. A cleaner future design decouples them: boundaries at GpC-midpoints (sufficient, fast) + an independent fine dyad search per NUC segment.

7. Every v4 hack, re-expressed as model structure

This is the heart of why the rewrite is worth it:

v4 hand-coded penalty / behavior	v5 native mechanism
TF-boundary penalty (a TF must be flanked by accessible DNA)	falls out of emissions: a TF followed by an OPEN linker scores well only if the linker GpCs read accessible; a protected flank is naturally penalized by OPEN’s low <code>p_U</code>
“usually-but-not-always accessible between adjacent TFs”	TF $\rightarrow$ OPEN(linker) $\rightarrow$ TF: the OPEN linker <i>prefers</i> accessible bases but <i>tolerates</i> a protected one at a normal emission cost — a soft expectation, not a rule

v4 hand-coded penalty / behavior	v5 native mechanism
run-length Occam cap (long protected run -> nucleosome)	NUC duration prior + <code>start_cost</code> make a genuine nucleosome cheaper than a long TF run
di-nucleosome / dead-zone (130-270 bp) handling	NUC->NUC transition: two adjacent NUC segments, dyads at their own centers, linker-or-not decided by the between-dyad GpCs
parsimony (don't over-call footprints)	<code>start_cost</code> per segment
"collapse observationally-equivalent microstates"	not needed — there's no global microstate list to collapse
nucleosome/TF non-overlap pruning	segments tile the molecule by construction; they <i>can't</i> overlap
promoter-specific open probability	optional (<code>--promoter_positions</code>), off by default now — with UNID we call footprints uniformly across the molecule

Guiding principle (from the spec): *if you find yourself wanting to add a special-case penalty, ask whether it belongs in the emission / duration / transition structure instead.*

8. Fitting the parameters

Right now: fixed parameters. All the numbers (conversion rates, `p_U` per state, nucleosome footprint width, duration prior, start/transition costs) are set by hand / config and held fixed. Decoding is a single Viterbi pass. This is deliberate — we want the structure trusted before adding a fitting loop.

Later: EM (`--do_em`, not yet built). The natural extension is Expectation-Maximization: 1. **E-step:** decode all molecules with the current parameters (Viterbi, or forward-backward for soft assignments). 2. **M-step:** re-estimate emission parameters from the decoded segments — e.g. take all GpCs assigned to TF segments, compute their observed protected-fraction, and invert the conversion model (`invert_t_fraction`, reused from v4) to recover `prob_unmeth_given_tf`; similarly fit the nucleosome sigmoid (`d_edge`, `softness`) to GpCs assigned to NUC segments by their distance to the dyad. 3. Repeat until the parameters stop moving.

`ModelParams` is structured so this drops in without reshaping the model. Structural costs (start/transition) are more like priors and would likely stay hand-set or be cross-validated rather than EM-fit.

9. What you get out, and how to read it

- **Tidy path** (`*.segments.txt`): one row per segment — `read_id`, `seg_index`, `type`, `start`, `end`, `dyad_or_motif`. This *is* the Viterbi path in state/bp space.
- **Wide per-molecule table** (`*.single_molecule_classification.v5.txt`): `tfbs_{k}` binary calls (which TetOs are bound), `n_nuc/n_unid`, semicolon lists of nucleosome (`start:end:dyad`) and UNID (`start:end`) intervals, and the path `log_likelihood`.
- **Plots:** a bulk sanity page (observed data vs predicted state, 1000 reads, 4 colors, aligned on the bp axis) followed by individual per-read traces with the segmentation drawn on top (NUC gray, TF red, UNID purple).

10. Known limitations / things to keep in mind

- **A single global nucleosome footprint width** (`d_edge/softness` are one pair for all nucs). The duration prior gives per-nuc length flexibility, but the *emission* soft-edge shape is shared.

- **Grid resolution** (`grid_step`, default 5 bp): dyad/boundary placement is quantized to the grid (motif edges excepted). Fine for these amplicons; lower it for sharper boundaries at a quadratic speed cost.
- **UNID is intentionally narrow** (off-motif AND short). It will *not* flag an anomalous footprint that happens to sit on a motif (that region is reserved for TF) or one that is nucleosome-length (that's a NUC). This is a deliberate identifiability choice, revisit if the new data has off-length-on-motif footprints.
- **Structural costs are first-guesses.** The defaults are reasonable but not calibrated; the edge-case molecules we work through are how we'll tune `start_cost_*` and `trans_*`.
- **log-likelihoods are within-v5 only** — don't compare them to v4 or across very different parameter settings.